LMS Course List Pagination Plan

YurCore Learning · 27 October 2026 · Plan only, not yet implemented

Summary. Both LMS course lists load every course at once. Supabase returns at most 1,000 rows per query, so past roughly 1,000 courses, courses quietly go missing and content counts come out wrong even earlier. The plan loads one page of courses at a time and counts the totals in the database. No database migration is needed.

1. What pagination is

Pagination means loading a list one page at a time instead of all at once. The browser asks for "page 1, 24 courses". The server replies with just those 24, plus the total count so the screen can show "Page 1 of 42".

There are two common styles:

- **Page numbers** (`page=3&pageSize=24`): good for tables where you jump to a page.
- **Load more** (`limit` and `offset`): a button adds the next 24 under the current ones. Good for card grids.

In Supabase this is one call, `.range(from, to)`. With `{ count: "exact" }` added, the same query also returns the total.

2. What your project does today

Admin list (`/admin/learning`)

1. The page calls `GET /api/admin/learning/courses`.
2. The server loads **every** course, then **every** section and content item of every course (only to count them). It also makes a signed thumbnail link for every course, and loads every enrollment and the creator's email for each one.
3. The browser receives all of it, then filters by search, category and status in JavaScript. The 4 stat cards (Total, Drafts, Published, Learners) are counted from that full array.

Practitioner list (`/pro/learning`, plus My Learning and Completed)

1. `CourseList` calls `GET /api/learning/courses?scope=...&q=...&category=....`
2. The server already filters by search and category, but still returns every match, again with all sections, content and progress.
3. The "Completed" tab is filtered in JavaScript after everything has been loaded.

The problem: Supabase returns at most 1,000 rows per query. So past about 1,000 courses, courses quietly go missing. Even earlier, the content counts come out wrong, because the content query reaches 1,000 rows long before the course query does. Requests also get slow and large.

You already have a ready-made type for paginated responses, `PaginatedResponse<T>` in `src/types/api.types.ts`, with `page`, `pageSize` and `hasMore`. The plan reuses it.

3. What we'll implement, step by step

Step 1: Database query (src/backend/repo/learning.repo.ts)

- `listCourses` takes `page` and `pageSize`, uses `.range()`, and returns `{ rows, total }`.
- It also filters by `status` in the query, so the browser no longer has to.
- Add `listAllCourses()`, which reads in batches of 1,000. The Moodle sync needs every course, so it switches to this one.
- Add `courseStats()`: three small count queries for the admin stat cards, so they no longer depend on loading every course.

Step 2: Counts and thumbnails only for the visible page (src/backend/service/learning-common.ts)

- `summaries()` stays as it is, but now receives at most 24 courses. Sections, content and thumbnail links are fetched only for those 24, which stays far below the 1,000-row cap.
- No database migration is needed for this.

Step 3: Admin service and route (learning-admin.service.ts, api/admin/learning/courses)

- `LearningAdminService.list()` accepts `page`, `pageSize`, `q`, `category` and `status`. It returns `{ items, total, page, pageSize, hasMore, stats }`.
- The route reads these from the URL, for example `?page=2&pageSize=24&status=draft`.

Step 4: Practitioner service and route (learning.service.ts, api/learning/courses)

- Same paging for `LearningService.list()`, which returns `{ items, total, page, pageSize, hasMore }`.
- **The "Completed" tab becomes a database filter.** It first gets the IDs of courses where `completed_at` is set, then queries only those, so a page can never come back short.
- Progress is loaded only for the courses on that page.

Step 5: Admin page (src/app/admin/(app)/learning/page.tsx)

- Search, category and status are sent to the server instead of being filtered in the browser. Search waits briefly after typing, the same way the practitioner list already does.
- Add Previous and Next buttons with "Page X of Y" under the table.
- Stat cards use the server's `stats`.
- The category dropdown uses the existing `/categories` endpoint, since one page of courses no longer contains every category.
- The 4-second auto-refresh during Moodle syncs keeps working, for the current page.

Step 6: Practitioner page (src/app/(portal)/pro/learning/_components/CourseList.tsx)

- A "Load more" button adds the next 24 courses to the grid.
- Changing search, category or tab starts again from page 1.

Step 7: Check

- Type-check, lint and unit tests.
- Add one unit test for the page math (`from`/`to`, `hasMore`).
- I'd like to test with seeded data, for example 1,200 dummy courses. That needs your OK, because it writes to your Supabase database.

4. What it affects in the rest of the project

Area	Effect
The two course-list API responses	**Shape changes** from a plain list to `{ items, total, ... }`. Only the two LMS pages use them, and both get updated in the same change.
Course detail, player, quizzes, uploads, publish	No change. These work on one course and are already fast.
Moodle sync	Uses the new `listAllCourses()`. Same behaviour, and it also stops missing courses past 1,000.
Database	No migration needed.
Other modules (appointments, EMR, discover...)	No change. `useQuery` and `apiClient` stay as they are.
Users	Admins see a paged table; practitioners get "Load more". Pages load faster and counts are correct.

5. Left for later

- **Faster search beyond about 10,000 courses.** This needs a database migration (a `pg_trgm` text index).
- **Video streaming (HLS) or a CDN for long videos.** This is a separate project.